

more than just print `ls -l` exactly on the string. So the question is why did we use two blank `echo` commands in the middle? That was a simple way of inserting new line characters.

If you notice that `echo 'hello'`, `echo "hello"` and a `echo hello` seem to be producing the same results, so let us have a little detailed look at it since quotation mark are an extremely important and confusing feature of scripting. We will deal with the meta characters or special characters. Let us take the example where we want to use the `$` character as just the character and not with its special meaning.

This will be illustrated nicely by the following example

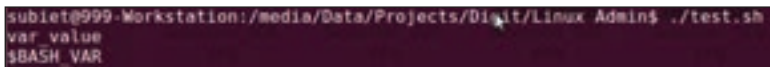
```
#!/bin/bash
```

```
#Declare bash string variable
var_name="var_value"
# echo variable var_name
echo $var_name
#when meta character such us "$" is escaped with "\"" it
will be read literally
echo \ $BASH_VAR
# Since \ is also a meta character its meaning can be
suppressed with another \
echo "\\ "
```

Single quotes can also be used to suppress the special meaning of any meta characters, so the above example can also be re-written with single quotes. It is not possible to use another single quote within two single quotes not even if the single quote is escaped by backslash.

```
#!/bin/bash
```

```
#Declare bash string variable
var_name="var_value"
# echo variable var_name
echo $var_name
#when meta character such us "$" is escaped with "\"" it
will be read literally
echo ' $BASH_VAR '
# Since \ is also a meta character its meaning can be
suppressed with another \
```



```
sublet@999-Workstation:/media/Data/Projects/Digit/Linux Admin$ ./test.sh
var_value
$BASH_VAR
\
```

The output of the code above